ShelfShare

Project Specification Document

Project: ShelfShare – A Neighborhood Lending Library
Demo Neighborhood: Maple Grove (Fictional)
Course: WDD430 – Web Full-Stack Development
Status: Draft for team review and Spec-Kit refinement

Team Members:

- Gwyn Wright
- Daniel Rasmuson
- Danae De la Cruz J.
- Emmanuel Nduka Eze

---

1. Project Title and Description

ShelfShare is a community lending library web application that allows neighbors to lend and borrow everyday household items rather than purchasing things they may only need occasionally.

Users can create accounts, list items available for borrowing, browse other residents' listings, submit borrowing requests, and manage their lending activities through a personal dashboard.

For development and demonstration, ShelfShare will use a fictional neighborhood called **Maple Grove**, with sample residents and household items. This allows the team to demonstrate the application's features without using real residents' personal information.

2. Purpose and Target Audience

Purpose

ShelfShare aims to help communities save money, reduce unnecessary purchases, decrease waste, and encourage cooperation among neighbors.

Target Audience

- College students living on or near campus

- Apartment residents
- Neighborhood communities
- Renters and homeowners
- Community organizations interested in resource sharing

Problem Being Solved

People frequently purchase tools, camping equipment, kitchen appliances, and other products they rarely use. ShelfShare provides an organized way for community members to share these items.

3. Core Features

Feature 1: User Accounts and Profiles

Users can register, sign in, sign out, and manage their profiles.

Profile information includes:

- Name
- Email address
- Neighborhood
- Short biography

Authentication will use Auth.js.

Feature 2: Item Listings (CRUD)

Registered users can create, read, update, and remove their own item listings.

Each listing includes:

- Item title
- Description
- Category
- Condition
- Photo
- Availability
- Owner information

Removing an item uses a soft delete, meaning the listing is marked unavailable rather than permanently deleted from the database.

Feature 3: Browse and Search

Users can browse a responsive catalog of items available in Maple Grove.

Search features include:

- Keyword search
- Category filtering
- Item detail pages
- Availability indicators

Feature 4: Borrow Requests (CRUD)

Users can request items from other residents for specific dates.

Borrowing features include:

- Submit a request
- Select borrowing dates
- View request status
- Edit or cancel a pending request
- Approve or decline requests as the item owner
- Mark borrowed items as returned

Feature 5: My Dashboard

Registered users have a personal dashboard containing three sections:

My Items: Listings created by the user.

Requests I've Sent: Items the user has requested to borrow.

Requests I've Received: Requests submitted by other users for the user's items.

Requests display appropriate statuses, including pending, approved, declined, canceled, and returned.

---

4. User Stories and Acceptance Criteria

User Story 1: Account Registration

As a new resident, I want to create an account so that I can participate in the neighborhood lending library.

Acceptance Criteria

- Given valid registration information, when a user registers, then an account is created.
- Given an email already in use, when registration is attempted, then an appropriate error message appears.
- Given an unauthenticated visitor, when they attempt a protected action, then they are directed to sign in.

Priority: P0

User Story 2: Create an Item Listing

As a registered user, I want to list items I am willing to lend so that my neighbors can borrow them.

Acceptance Criteria

- Users can enter an item title, description, category, condition, and availability.
- Users can attach a photo to their listing.
- Required fields are validated.
- Successfully saved listings appear in the catalog.
- Listings are associated with the authenticated owner.

Priority: P0

User Story 3: Browse and Search Items

As a resident, I want to browse and search available items so that I can find something I need.

Acceptance Criteria

- The catalog displays item cards with titles, categories, images, and availability.
- Users can filter items by category.
- Users can search by keyword.
- Users can open an individual item detail page.
- The catalog works on mobile and desktop screens.
- An appropriate message appears when no items match.

Priority: P0

User Story 4: Request an Item

As a borrower, I want to request an item for specific dates so that I can borrow it when needed.

Acceptance Criteria

- Signed-in users can submit requests for available items.
- Requests include start and end dates.

- The system rejects invalid date ranges.
- Users cannot request their own items.
- New requests receive a pending status.
- Borrowers can edit or cancel pending requests.

Priority: P0

User Story 5: Manage Borrow Requests

As an item owner, I want to approve or decline borrowing requests so that I can control when my items are loaned.

Acceptance Criteria

- Owners can view requests for their items.
- Owners can approve or decline pending requests.
- Borrowers can see updated request statuses.
- Only the item owner can approve or decline requests.
- The system prevents conflicting approved borrowing dates.

Priority: P0

User Story 6: Manage My Listings

As an item owner, I want to edit or remove my listings so that the catalog contains accurate information.

Acceptance Criteria

- Owners can update their own item information.
- Owners can mark listings unavailable.
- Unavailable listings cannot receive new borrowing requests.
- Other users cannot edit or remove another owner's listings.

Priority: P1

User Story 7: Personal Dashboard

As a registered user, I want to view my items and borrowing activity in one place so that I can manage my participation.

Acceptance Criteria

- Users can view their own listings.
- Users can view requests they have sent.
- Users can view requests they have received.

- Each request displays its current status.
- Dashboard information is restricted to the signed-in user.

Priority: P1

---

5. Technical Requirements

Framework: Next.js with App Router
Language: TypeScript with strict mode
Styling: Tailwind CSS
Authentication: Auth.js
Database: PostgreSQL
ORM: Prisma
Version Control: Git and GitHub
Formatting: ESLint and Prettier
Deployment: Vercel

Development Standards

- Use TypeScript interfaces and types rather than `any`.
- Use server components by default.
- Use client components when interactive browser functionality is required.
- Use Tailwind CSS utilities for styling.
- Use descriptive component and variable names.
- Keep sensitive credentials in environment variables.
- Validate user input on the server.
- Verify authentication and ownership before protected database operations.
- Use feature branches and reviewed pull requests.

---

6. Database Design

User

Field	Description
id	Unique user identifier
name	Resident name
email	Unique email

neighborhood	Neighborhood name
bio	Short biography
createdAt	Account creation date

Auth.js will require additional authentication-related fields or tables depending on the selected provider.

Item

Field	Description
id	Unique item identifier
ownerId	Reference to owner
title	Item name
description	Item details
category	Item category
condition	Item condition
imageUrl	Item photo location
isAvailable	Availability indicator
isDeleted	Soft-delete indicator
createdAt	Creation date

BorrowRequest

Field	Description
id	Unique request identifier
itemId	Requested item
borrowerId	Requesting user
startDate	Borrowing start date
endDate	Borrowing end date

status	Pending, approved, declined, canceled, or returned
createdAt	Request creation date

Relationships

- One user can own many items.
- One user can submit many borrowing requests.
- One item can receive multiple borrowing requests.
- Each borrowing request belongs to one item and one borrower.

---

7. Core API Endpoints

Method	Endpoint	Purpose
GET	`/api/items`	Retrieve item listings
GET	`/api/items/[id]`	Retrieve item details
POST	`/api/items`	Create a listing
PATCH	`/api/items/[id]`	Update or soft-delete a listing
GET	`/api/requests`	Retrieve the current user's requests
POST	`/api/requests`	Submit a borrowing request
PATCH	`/api/requests/[id]`	Edit, cancel, approve, decline, or return a request
GET	`/api/profile`	Retrieve the current user's profile
PATCH	`/api/profile`	Update profile information

Auth.js will handle authentication routes through its configured Next.js integration.

Protected endpoints must verify the current user and the user's authorization to access or modify the requested resource.

---

8. Fictional Neighborhood and Sample Data

Neighborhood Name: Maple Grove

Maple Grove is a fictional residential community used for testing and demonstrating ShelfShare.

Example Residents

Resident	Example Item
Alex Morgan	Cordless drill
Jamie Parker	Camping tent
Taylor Brooks	Stand mixer
Casey Rivera	Gardening tools

Additional sample items can include books, ladders, board games, bicycles, and small kitchen appliances.

These records are fictional and will be used to test item browsing, ownership, and borrowing workflows.

---

9. Implementation Priorities

P0 — Essential Features

- Next.js application setup
- Database configuration
- Authentication
- User profiles
- Create and view item listings
- Browse and search
- Submit borrowing requests
- Approve and decline requests

P1 — Important Features

- Edit and soft-delete listings
- Edit and cancel pending requests
- Personal dashboard
- Return tracking

- Improved mobile responsiveness and accessibility

P2 — Future Enhancements

- Multiple neighborhood support
- User ratings and reviews
- Email notifications
- Community messaging
- Map-based item discovery

P2 features are outside the initial required implementation and should only be considered after essential features are complete.

---

10. Proposed Team Responsibilities

Assignments are subject to team approval.

Gwyn Wright — Authentication and Setup

- Repository initialization
- Authentication and user profiles
- Collaboration and repository configuration

Daniel Rasmuson — Item Management

- Item database model
- Listing forms
- Item CRUD operations
- Item detail pages

Danae De la Cruz J. — Catalog and Design

- Homepage
- Browse and search functionality
- Responsive layout
- Accessibility and shared UI components

Emmanuel Nduka Eze — Borrowing and Dashboard

- Borrow request workflows
- Approval and decline functionality
- Request status management
- User dashboard

All team members participate in specification review, testing, pull request reviews, and integration.

---

11. Testing and Quality Assurance

The team will test the following:

- Account registration and authentication
- Protected pages and API endpoints
- Item creation, editing, and soft deletion
- Search and category filters
- Borrow request submission and approval
- Invalid dates and conflicting requests
- Dashboard information and ownership permissions
- Responsive layouts on mobile and desktop
- Keyboard navigation and accessible form labels
- Lighthouse Performance, Accessibility, Best Practices, and SEO

---

12. GitHub Collaboration Guidelines

- The `main` branch will be protected against direct pushes.
- Each feature will be developed on a separate branch.
- Pull requests are required before merging.
- At least one other team member must review each pull request.
- GitHub Issues will track development tasks.
- The GitHub Project Board will contain at least seven issues.
- Each issue should include a description, acceptance criteria, priority, and assignee when known.

---

13. Definition of Done

A feature is complete when:

1. It meets its documented acceptance criteria.
2. The application builds without errors.
3. Relevant functionality has been tested.
4. Responsive design and accessibility have been checked.

5. Code has been reviewed through a pull request.
6. The feature has been merged into `main`.

---

14. Project Success Criteria

ShelfShare will be considered successful when users can create accounts, list household items, browse available listings, request to borrow items, approve or decline requests, and manage their activity through a dashboard.

The application must demonstrate these workflows using the fictional Maple Grove neighborhood, with appropriate authentication, data validation, responsive design, and database persistence.